Rapport OpenMP

Jacques Gastebois

16 mai 2025

Table des matières

1 Introduction **2**

2 Configuration de l'Environnement de Compilation **2**

3 Code Initial et Fonctions de Base **3**
3.1 En-têtes (Includes) 3
3.2 Fonction `exercice1` - Hello World Parallèle 3
3.3 Fonction `exercice2` - Boucle Parallèle Simple 3
3.4 Fonction `exercice3` - Gestion des Chunks 4

4 Optimisation de la Multiplication de Matrices (`exercice4`) **5**
4.1 `exercice4` (v0) - Version Initiale 5
4.2 `exercice4_v1_no_loop_printf` (v1) - Suppression des `printf` 6
4.3 `exercice4_v2_loop_interchange` (v2) - Interversion des Boucles 6
4.4 `exercice4_v3_collapse` (v3) - Clause `collapse` 6
4.5 `exercice4_v4_simd` (v4) - Vectorisation SIMD 6
4.6 `exercice4_v5_unroll` (v5) - Déroulage Manuel de Boucle 7
4.7 `exercice4_v6_dynamic_sched` (v6) - Ordonnancement Dynamique 7
4.8 Tableau Récapitulatif des Performances (`exercice4`, NR=500, 8 threads) 7

5 Conclusion **7**

1 Introduction

L'objectif de ce document est de détailler la démarche d'optimisation d'un programme de multiplication de matrices en C, en utilisant la bibliothèque OpenMP pour la parallélisation. Les tests et compilations sont effectués sur un environnement spécifique : un Apple MacBook Pro équipé d'un processeur M2 Max (architecture ARM64). Ce rapport couvrira la configuration initiale de l'environnement de compilation, le code de base, les différentes étapes d'optimisation de la fonction de multiplication matricielle, et une synthèse des performances observées. La taille de matrice utilisée pour les tests d'optimisation principaux est `NR=500` avec 8 threads OpenMP.

2 Configuration de l'Environnement de Compilation

La machine cible est un Mac M2 Max. Initialement, la compilation C utilisant OpenMP se heurtait à des problèmes liés à l'architecture. Homebrew, installé pour l'architecture x86_64 (probablement via Rosetta 2), fournissait une version de 'libomp' incompatible avec une compilation native ARM64.

Les étapes suivantes ont été suivies pour rectifier la situation :

1. Identification de l'installation native ARM64 de Homebrew dans '/opt/homebrew'.
2. Désinstallation de la version x86_64 de 'libomp'.
3. Installation de la version ARM64 de 'libomp' via le Homebrew natif.
4. Modification du 'Makefile' pour prendre en charge la compilation ARM64 native et permettre le choix de l'architecture via une variable 'ARCH'. Les chemins d'inclusion et de liaison pour 'libomp' ont été ajustés en conséquence.

Ces ajustements ont permis de compiler et d'exécuter le programme nativement sur l'architecture ARM64 de la puce M2 Max. Un effet notable de cette transition vers une compilation et exécution ARM64 native a été une **diminution significative du nombre de context switches** observés durant l'exécution du programme (mesurés via 'getrusage'). Cela suggère une meilleure adéquation entre le programme et le système d'exploitation, ainsi qu'une réduction des surcoûts potentiels liés à l'émulation ou à la gestion de bibliothèques d'architectures différentes.

Le 'Makefile' a été adapté comme suit (extrait pertinent) :

```makefile
# Makefile

# Compilateur C
CC = clang

# Options de compilation de base
CFLAGS_COMMON = -Wall -Wextra -std=c11 -g

# Definition de l'architecture (arm64 ou x86_64)
# Peut etre surcharge en ligne de commande: make ARCH=x86_64
ARCH ?= arm64

# Options specifiques a OpenMP et a l'architecture
ifeq ($(ARCH), arm64)
    OMP_CFLAGS = -Xpreprocessor -fopenmp -I/opt/homebrew/opt/libomp/include
    OMP_LDFLAGS = -L/opt/homebrew/opt/libomp/lib -lomp
    TARGET_ARCH = -arch arm64
else ifeq ($(ARCH), x86_64)
    OMP_CFLAGS = -Xpreprocessor -fopenmp -I/usr/local/opt/libomp/include
    OMP_LDFLAGS = -L/usr/local/opt/libomp/lib -lomp
    TARGET_ARCH = -arch x86_64
else
    $(error Architecture $(ARCH) non supportee. Utiliser arm64 ou x86_64)
endif

# CFLAGS finaux, incluant les optimisations
CFLAGS = $(CFLAGS_COMMON) $(OMP_CFLAGS) $(TARGET_ARCH) -O3 -ffast-math -flto=thin
LDFLAGS = $(OMP_LDFLAGS) -flto=thin
```

```

# Cible principale
TARGET = hello_world
# ... (reste du Makefile)
```

Listing 1 – Extrait du Makefile modifié

Des options de compilation supplémentaires telles que ‘-O3‘, ‘-ffast-math‘ et ‘-flto=thin‘ ont été ajoutées ultérieurement pour tenter d’améliorer encore les performances.

3 Code Initial et Fonctions de Base

3.1 En-têtes (Includes)

```c
#include <omp.h> // indique au compilateur que l'on utilise la bibliotheque OpenMP pour paralelliser les instruction
#include <stdio.h> // indique au compilateur que l'on utilise la bibliotheque d'entree sortie standard
#include <stdlib.h> // pour atoi et getenv
#include <sys/resource.h> // pour getrusage
#include <string.h> // pour strcmp, etc.
```

Listing 2 – En-têtes du programme C

3.2 Fonction `exercice1` - Hello World Parallèle

Cette fonction est une introduction à OpenMP, créant une région parallèle où chaque thread s’identifie.

```c
// Exercice 1 - Hello World
void exercice1() {
    #pragma omp parallel // definie une section de code    paralelliser
    {
        printf("Hello world! I'm thread %d out of %d threads.\n",
        omp_get_thread_num(), omp_get_num_threads()); // affiche le message hello worlds et les informations du thread courant sur la sortie standard.
        // Par defaut, on utilise tous les coeurs disponibles sur la machine (12 sur la mienne) mais on peut changer ca en jouant sur
        // la variable d'environement OMP_NUM_THREADS ou en codant en dur omp_set_num_threads();
    }
}
```

Listing 3 – Fonction exercice1

3.3 Fonction `exercice2` - Boucle Parallèle Simple

Distribue les itérations d’une boucle simple entre les threads.

```c
// Exercice 2 - Parallel For
void exercice2() {
    int n; // initialise
    #pragma omp parallel for // indique une boucle for a paralleliser
    for (n = 0; n<8; n++){ // on incremente n de 0 a 7
        printf("parallel thread %d iteration %d\n",
        omp_get_thread_num(), n);
    } //affiche le numero de thread
}
```

Listing 4 – Fonction exercice2

3.4 Fonction exercice3 - Gestion des Chunks

Explore les stratégies de distribution d'itérations 'schedule(static,chunk)' et 'schedule(dynamic,chunk)'.

```c
// Exercice 3 - Chunks
#define CHUNKSIZE 10  // Avec un chuncksize de 10 pour 100 iterations sans contrainte en therme de thread,
#define N 120         //  on voit  12 threads demarrer mais seulemt 10 executer la boucle avec 10 iterations par thread.

void exercice3() {
    int nthreads, tid, i, chunk;
    float a[N], b[N], c[N];

    /* initialisation des tableaux */
    for (i=0; i < N; i++)
        a[i] = b[i] = i * 1.0;

    chunk = CHUNKSIZE;

    // on definit une portion de code a paralelliser en indiquant
    // les variable accessibles par tous les threads et celle qui leurs sont propres (i,tid)
    #pragma omp parallel shared(a,b,c,nthreads,chunk) private(i,tid)
    {
        tid = omp_get_thread_num(); // stock l'identifiant du thread
        if (tid == 0) { // s'il s'agit du thread 0, on affiche le nombre total de thread qui vont etre executé s,
                        //  comme l'ordre d'execution des threads n'est pas sequentiel, il faut chercher au milieu des infos afficher par le terminal pour trouver l'info (ce qui n'est pas tres pratique)
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

        printf("Thread %d starting...\n", tid);

        #pragma omp for schedule(static,chunk) // lance en parallele la meme instruction en indiquant
                                                // que chaque thread doit executer X iterations chacun (X=Chunks)
                                                // et de facon static, c'est a dire que chaque iterations est assigné a un thread avant execution
        for (i=0; i<N; i++) {
            c[i] = a[i] + b[i];  //  fait la sommes des elements de a et b de meme indice et on les stocks dans le tableau c
            printf("Thread %d: c[%d]= %f\n", tid, i, c[i]); // affiche l'identifiant du thread, l'indice et le contenue de la cellule qui vient d'etre remplie
        }
    }

    #pragma omp parallel shared(a,b,c,nthreads,chunk) private(i,tid)
    {
        tid = omp_get_thread_num(); // stock l'identifiant du thread
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

        printf("Thread %d starting...\n", tid);

        #pragma omp for schedule(dynamic,chunk) // lance en parallele la meme instruction en indiquant
                                                // que chaque thread doit executer X iterations chacun (X=Chunks)
                                                // en dynamique cette fois ci, on attribut un thread a chaque iteration a la vol , des que le thread est
```

```
    disponible.
        for (i=0; i<N; i++) {
            c[i] = a[i] + b[i];  //  fait la sommes des elements de a et b de meme indice et on les stocks dans le tableau c
            printf("Thread %d: c[%d]= %f\n", tid, i, c[i]); // affiche l'identifiant du thread, l'indice et le contenue de la cellule qui vient d'etre remplie
        }
    }
}
```

Listing 5 – Fonction exercice3

4 Optimisation de la Multiplication de Matrices (`exercice4`)

La fonction ‘exercice4‘ a fait l'objet de plusieurs itérations d'optimisation. La taille de la matrice a été fixée à `NR=500` et les tests ont été principalement conduits avec 8 threads OpenMP.

4.1 `exercice4` (v0) - Version Initiale

Cette version de base implémente la multiplication matricielle parallélisée, mais inclut des ‘printf‘ dans les boucles de calcul, ce qui la rend très lente.

```
#define NR 500

void exercice4(){
    int tid, nthreads, i, j, k, chunk;
    double a[NR][NR], /* matrix A to be multiplied */
    b[NR][NR], /* matrix B to be multiplied */
    c[NR][NR]; /* result matrix C */
    chunk = 10; /* set loop iteration chunk size */

    #pragma omp parallel shared(a,b,c,nthreads,chunk) private(tid,i,j,k)
    {
    tid = omp_get_thread_num();
    if (tid == 0)
    {
        nthreads = omp_get_num_threads();
        printf("Starting matrix multiple example with %dthreads\n",nthreads);
        printf("Initializing matrices...\n");
    }
    #pragma omp for schedule (static, chunk)
    for (i=0; i<NR; i++) // boucle sur les lignes de la matrice a et b
        for (j=0; j<NR; j++) // boucle sur les colonnes de la matrice a et b
            a[i][j]= i+j; // remplit la matrice a avec la somme des indices
    #pragma omp for schedule (static, chunk)
    for (i=0; i<NR; i++)
        for (j=0; j<NR; j++)
            b[i][j]= i*j; // remplit la matrice b avec le produit des indices
    #pragma omp for schedule (static, chunk)
    for (i=0; i<NR; i++)
        for (j=0; j<NR; j++)
            c[i][j]= 0; // initialise la matrice resultat a zero
    printf("Thread %d starting matrix multiply...\n",tid); // affiche le debut du calcul pour chaque thread
    #pragma omp for schedule (static, chunk)
    for (i=0; i<NR; i++) // boucle sur les lignes de la matrice resultat
    {
        printf("Thread=%d did row=%d\n",tid,i); // affiche le thread qui calcule la ligne i
        for(j=0; j<NR; j++) // boucle sur les colonnes de la matrice resultat
            for (k=0; k<NR; k++) // boucle sur les elements pour le produit scalaire
                c[i][j] += a[i][k] * b[k][j]; // calcule la somme des produits pour chaque element de la matrice resultat
    }
```

```c
    }
    printf("*******************************************************\n");
    printf("Result Matrix:\n");
    for (i=0; i<NR; i++)
    {
        for (j=0; j<NR; j++)
            printf("%6.2f ", c[i][j]); // affiche chaque element de la matrice resultat
        printf("\n"); // retour a la ligne apres chaque ligne de la matrice
    }
    printf("*******************************************************\n");
    printf ("Done.\n");
}
```

Listing 6 – Fonction exercice4 (v0 - Version Initiale avec commentaires originaux)

Performance (v0) : Très lente, dominée par les E/S des ‘printf‘ dans la boucle principale. Sert de point de départ avant optimisation.

4.2 `exercice4_v1_no_loop_printf` (v1) - Suppression des `printf`

La première optimisation a consisté à supprimer les ‘printf‘ à l'intérieur des boucles de calcul intensif.

- **Modification** : Retrait du ‘printf("Thread=
- **Performance (v1)** : Amélioration très notable par rapport à v0. Le temps de calcul devient mesurable et n'est plus masqué par les E/S.

4.3 `exercice4_v2_loop_interchange` (v2) - Interversion des Boucles

L'ordre des boucles de multiplication a été changé de ‘i,j,k‘ à ‘i,k,j‘ pour améliorer la localité des données en cache, en particulier pour l'accès à ‘b[k][j]‘. La parallélisation OpenMP (‘pragma omp for schedule(static, chunk)‘) est appliquée à la boucle externe ‘i‘. Différentes tailles de ‘chunk‘ ont été testées.

- **Modification** : Boucles ‘for(i)... for(k)... for(j)...‘. La clause ‘schedule(static, chunk)‘ sur la boucle ‘i‘ avec ‘chunk=6‘ s'est avérée la plus performante.
- **Performance (v2)** : Temps d'exécution d'environ **0.0028 secondes** (pour NR=500, 8 threads, chunk=6). C'est notre nouvelle référence.

4.4 `exercice4_v3_collapse` (v3) - Clause `collapse`

Tentative d'utiliser ‘pragma omp for collapse(n)‘ pour paralléliser plusieurs boucles imbriquées.

- **Modifications** :
  - ‘collapse(2)‘ sur les boucles ‘i,k‘ (maintenant externes avec l'ordre ‘i,k,j‘).
  - ‘collapse(3)‘ sur les boucles ‘i,k,j‘.
- **Performance (v3)** :
  - ‘collapse(2)‘ : Moins performant que v2.
  - ‘collapse(3)‘ : Significativement plus lent que v2. L'ajout de ‘schedule(static, chunk)‘ (avec diverses valeurs de chunk) n'a pas permis de retrouver les performances de v2. La granularité fine et le surcoût de la parallélisation sur trois boucles semblent préjudiciables ici.

4.5 `exercice4_v4_simd` (v4) - Vectorisation SIMD

Ajout de ‘pragma omp simd‘ à la boucle la plus interne (‘j‘ dans la version v2 ‘i,k,j‘) pour encourager la vectorisation par le compilateur.

- **Modification** : ‘pragma omp simd‘ avant la boucle ‘for(j)...‘.
- **Performance (v4)** : N'a pas amélioré les performances par rapport à v2, voire les a légèrement dégradées. Cela suggère que le compilateur (Clang avec ‘-O3‘) effectuait déjà une vectorisation efficace de cette boucle, et la directive explicite n'apportait pas de gain ou introduisait un léger surcoût.

4.6 `exercice4_v5_unroll` (v5) - Déroulage Manuel de Boucle

Déroulage manuel (par un facteur de 4) de la boucle la plus interne (‘j‘ dans la version v2) pour réduire le surcoût de la boucle et potentiellement aider la localité et la vectorisation.

- **Modification** : Remplacement de la boucle ‘for(j)...‘ par une boucle déroulée traitant 4 éléments à la fois, avec une boucle résiduelle pour les éléments restants si ‘NR‘ n’est pas un multiple de 4.
- **Performance (v5)** : A dégradé les performances par rapport à v2. Le compilateur moderne est souvent meilleur pour décider quand et comment dérouler les boucles, et un déroulage manuel peut interférer avec ses propres optimisations.

4.7 `exercice4_v6_dynamic_sched` (v6) - Ordonnancement Dynamique

Test de ‘schedule(dynamic, chunk)‘ sur la boucle ‘i‘ de la version v2 (‘i,k,j‘), en remplacement de ‘schedule(static, chunk=6)‘.

- **Modifications** :
  - ‘schedule(dynamic, 1)‘ (ou ‘schedule(dynamic)‘ par défaut).
  - ‘schedule(dynamic, chunk)‘ avec des chunks plus grands (2, 4, 6).
- **Performance (v6)** :
  - ‘schedule(dynamic, 1)‘ : A donné des performances très compétitives, voire marginalement meilleures que ‘schedule(static, chunk=6)‘ de la v2, avec un temps d’environ **0.0027 secondes**.
  - Des chunks plus grands pour ‘dynamic‘ (2, 4, 6) se sont avérés moins performants que ‘chunk=1‘. L’ordonnancement dynamique avec un petit chunk est bénéfique ici, probablement en raison d’une meilleure répartition de la charge entre les threads, surtout si les itérations ont des temps d’exécution légèrement variables ou si l’environnement d’exécution fluctue (par exemple, gestion de l’énergie sur batterie).

4.8 Tableau Récapitulatif des Performances (`exercice4`, NR=500, 8 threads)

Table 1 – Performances des différentes versions de `exercice4`

Version	Optimisation Principale	Temps d’exécution (approx.)
v0	Version initiale avec `printf`	Très lent (secondes)
v1	Suppression des `printf` en boucle	Amélioration notable
v2	Boucles (`i,k,j`), `static, chunk=6`	**0.0028 s**
v3	`collapse(2)` / `collapse(3)`	Moins performant / Plus lent
v4	`simd` sur boucle interne (j)	Pas d’amélioration
v5	Déroulage manuel boucle (j)	Dégradation
v6	Boucles (`i,k,j`), `dynamic, chunk=1`	**0.0027 s**

Note : Les temps sont approximatifs et basés sur les observations lors des sessions de test. Ils peuvent varier légèrement en fonction de la charge système et d’autres facteurs.

5 Conclusion

La démarche d’optimisation a permis d’améliorer significativement les performances de la multiplication de matrices. La configuration correcte de l’environnement pour une compilation ARM64 native a été une étape préliminaire cruciale, réduisant notamment les changements de contexte.

Parmi les techniques OpenMP testées sur ‘exercice4‘ (avec ‘NR=500‘, 8 threads) :

- La suppression des E/S dans les boucles critiques (v1) a eu l’impact le plus drastique.
- L’interversion des boucles pour améliorer la localité du cache (‘i,k,j‘ - v2) a fourni un gain substantiel.

— L'ordonnancement dynamique (‘schedule(dynamic, 1)‘ - v6) sur la boucle externe parallélisée s'est avéré légèrement plus performant que l'ordonnancement statique optimisé, atteignant le meilleur temps de ~**0.0027 secondes**.
— Les directives ‘collapse‘, ‘simd‘ explicite et le déroulage manuel n'ont pas apporté de bénéfices dans ce contexte spécifique, le compilateur semblant déjà effectuer un travail d'optimisation poussé.

Ces résultats soulignent l'importance de comprendre les accès mémoire et de choisir judicieusement les stratégies de parallélisation et d'ordonnancement, tout en s'appuyant sur les capacités d'optimisation du compilateur moderne.

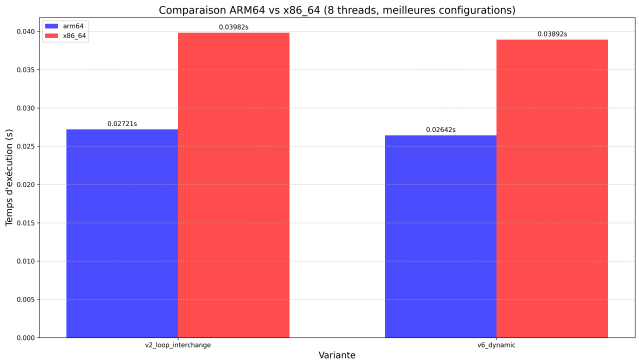

FIGURE 1 –

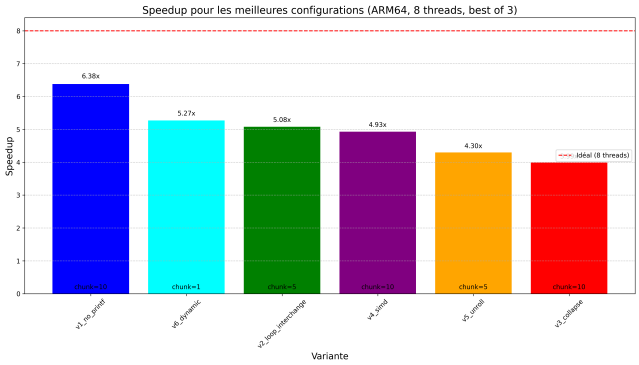

FIGURE 2 – Speed up = Temps d'exécution séquentiel / Temps d'exécution parallèle

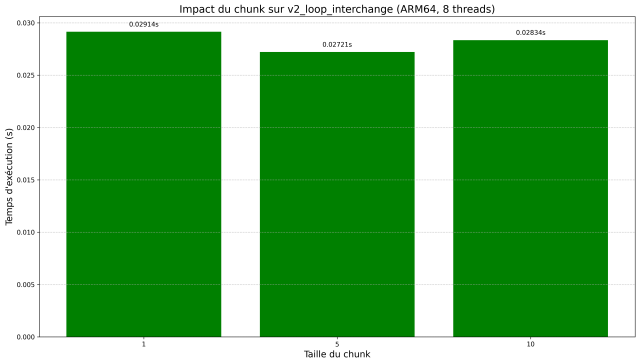

FIGURE 3 –

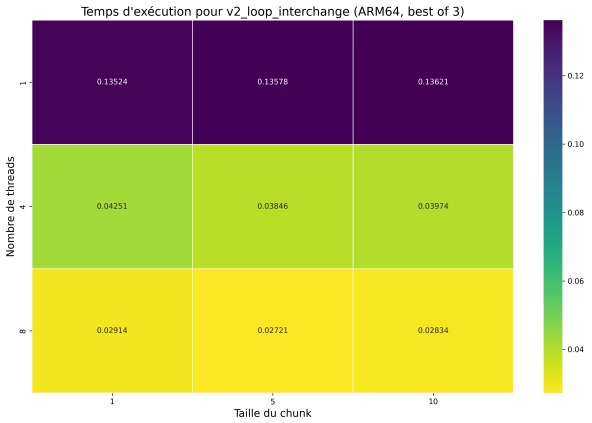

FIGURE 4 –

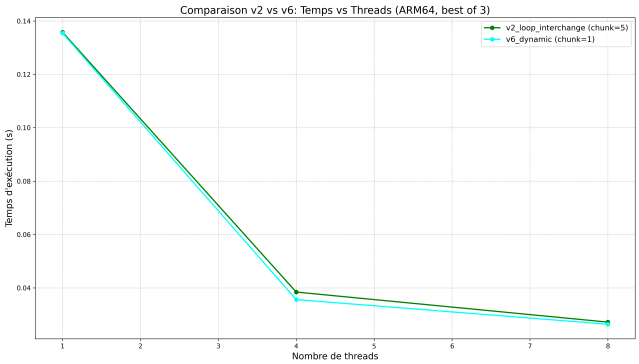

FIGURE 5 –

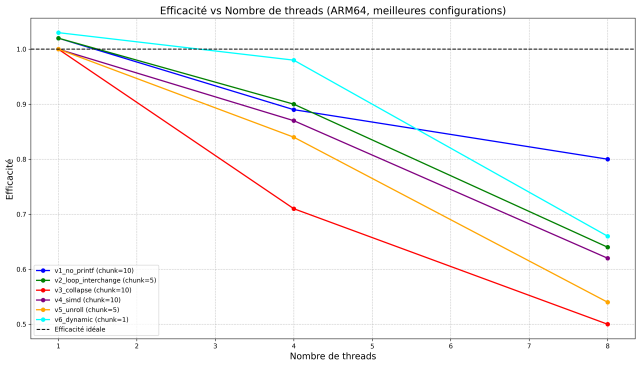

FIGURE 6 – Efficacité = Speed up / Nombre de Thread